

ایمان باباجانی

مدارمنطقی پروژه

این پروژه در رابطه با vending machine می باشد که دارای 6 استیت می باشد به ترتیب

1: IDLE

2: AVAILABLE

3: WAIT_COIN

4: CHECK_AMOUNT

5: DISPENSE

6: GIVE_CHANGE

1: این بخش، بخش ابتدای و بخشی است که مقدار پول کل به عدد صفر، نوع محصول در حالت نامعلوم، مقدار پولی که باید بازگشته شود به مقدار صفر میباشد.

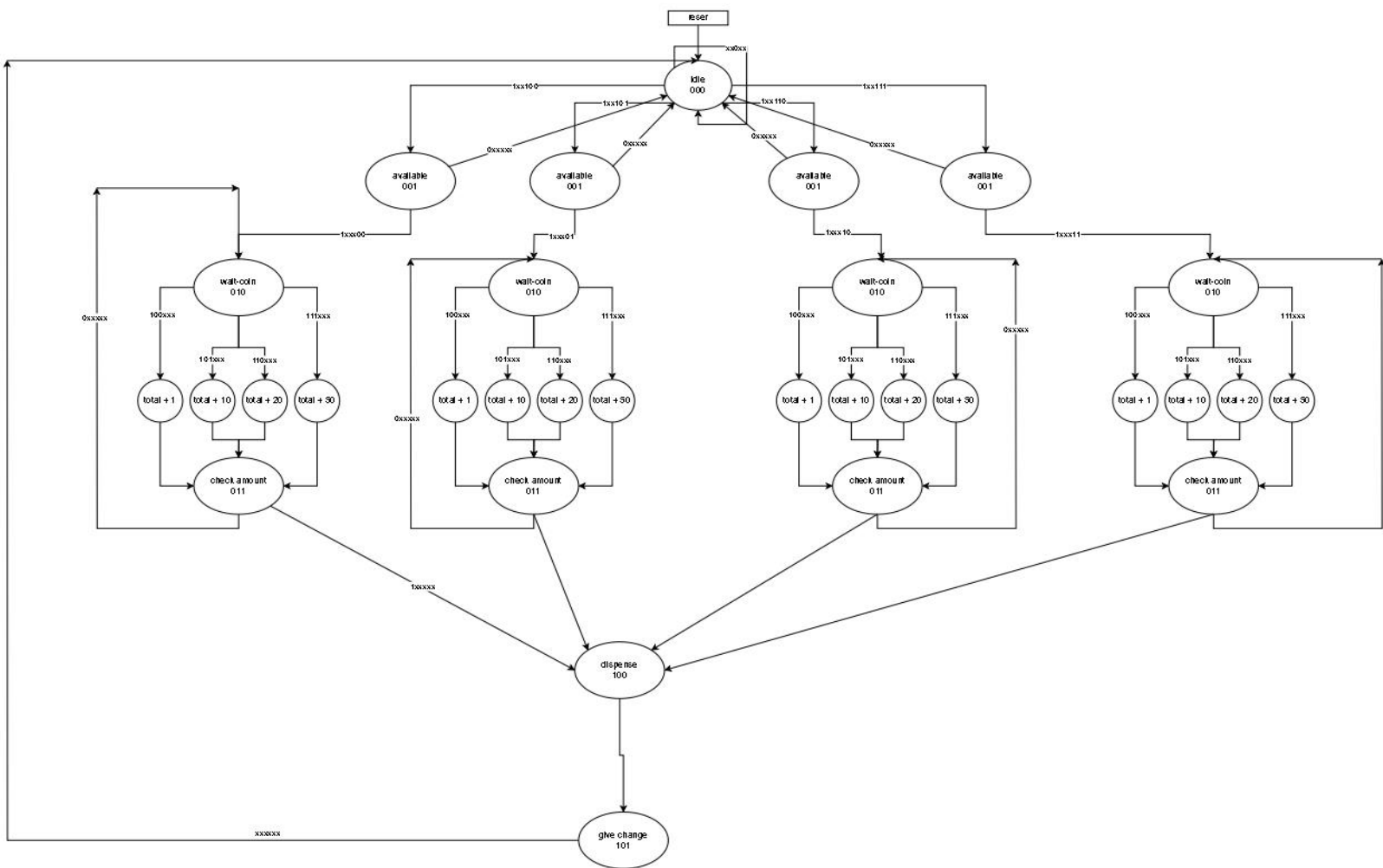
2: در این بخش چک می شود که در خود ماشین آیا جنس موجود هست یا نه. به (به صورت کلی استیت ارور من ندارم به اروری برخورد نمیکنم اگر نباشه صرفا دارم بین استیت اول و دوم میرم میام).

3: در این بخش ماشین منتظر می ماند تا فرد مورد نظر سکه بی اندازه به این شکل که سکه مدنظر که در ماشین قرار گرفت ماشین چک می کند از چه نوع سکه ای هست سپس به مقدار کل اضافه می کند مقدار مدنظر را در این بخش ارور و مشکل نبود پول کافی در نظر گرفته شده به صورتی که اگر پولی قرار نگیرد صرفا در این بخش می ایستد تا زمانی که پول کافی برای محصول مدنظر انتخاب شود

4: بخش چهار با بخش ۳ رابطه مستقیم دارد و دقیقا عملیات ایستادن برای پول کافی را اجرا می کند.

5: در این بخش در صورتی کل پول کافی یا به اندازه بود کالای مدنظر برای فرد به بیرون پرت شده و از تعداد اون محصول کم میشه.

6: این بخش، بخشی است که مقدار پولی که باید برگردانده شود به فرد داده می شود در صورتی که از قیمت محصول بیشتر باشد



State machine:

استیت ماشین به صورتی کشیده که برای هر کالا متفاوت نشان داده شود ولی در کل فقط ۶ استیت داریم

توضیحات لازم :

روی یال ها اعدادی قرار دارد که نشان می دهد در چه صورتی وارد استیت بعد می شویم

اگر ۶ ستاره را در نظر بگیریم

ستاره اول از سمت چپ نشان می دهد که استیت قبل کار خود را انجام داده.

۲ ستاره بعدی نشان می دهد چه نوع سکه ای در ماشین قرار گرفته

ستاره وسط نشان می دهد که آیا کاربر دکمه vend برای شروع کار را زده یا نه.

دو ستاره آخر نشان می‌دهد چه نوع کالایی انتخاب شده.

اگر x به جای ستاره قرار گرفته یعنی مهم نیست که چه اتفاقی رخ دهد ما در هر صورت عملیات را انجام می‌دهیم.

Verilog:

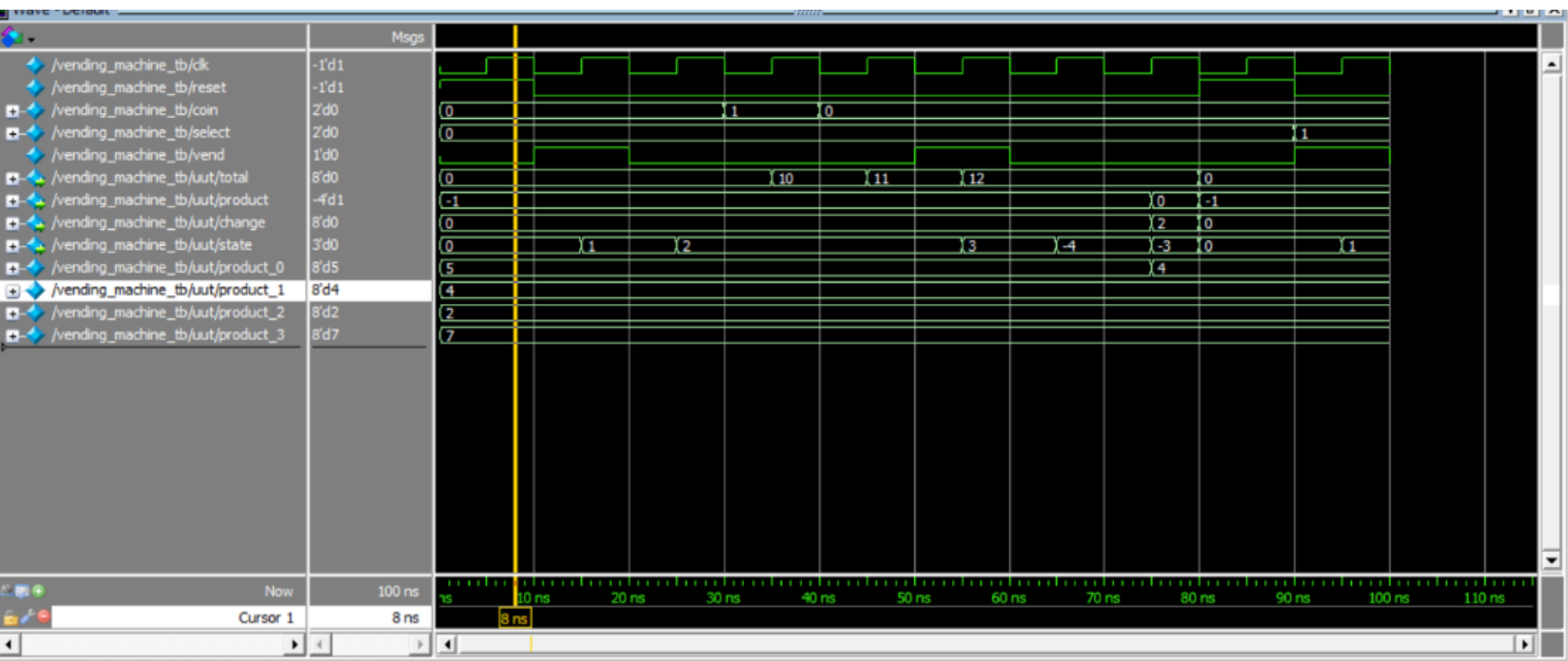
```
Transcript
# // is prohibited from disclosure under the Trade Secret Act,
# // 18 U.S.C. Section 1905.
# //
# Loading project vendinmachine
# Compile of vendingmachine.v was successful.
# Compile of vendingmachinetb.v was successful.
# 2 compiles, 0 failed with no errors.

ModelSim>
```

کامپایل موفق کد وریلاگ

۲ فایل مجموعاً وجود دارد یکی برای خود ماشین و دیگری برای تست ماشین

test:



توضیحات:

در تست یک بلاک برای خود کلاک گذاشته شده که در هر ۵ نانو ثانیه کلاک می‌زند مشاهده می‌شود که در ابتدا سیگنال ریست را فعال کرده که اتفاقی نی افتاد و به حالت ابتدایی بازمی‌گردد سپس با ۱ شدن سیگنال vend عملیات شروع شده و محصول مدنظر انتخاب شده در مراحل بعدی در کلاک های مختلف سکه های مختلف قرار می‌گیرند و تا زمانی که به قیمت مدنظر کالا (۱۰) ادامه می‌دهد وقتی که به میزان دلخواه رسید از تعداد کالا های که توسط خودم در ابتدای کد تعیین شده کم می‌شود و مقدار پول اضافه ای که مشتری قرار داده (۲) به او باز گردانده می‌شود.

در صورتی که مقدار کالا ها صفر شود در اسیت available سیستم گیر خواهید کد و بین استیت idle و available می‌ماند.

تبدیل به کد:

برای تبدیل استیت ماشین به کد ابتدا روی ورودی ها و خروجی ها فکر کردم ۲ بیت برای سکه ها در نظر گرفتن تا ۴ سکه را شامل بشه به ترتیب سکه ۱ کد ۰۰ سکه ۱۰ کد ۰۱ سکه ۲۰ کد ۱۰ و سکه ۵۰ کد ۱۱

برای استیت های ۳ بیت در نظر گرفتم و داخل خوده پروژه به عنوان پارامتر نوشتم

Idle 000

Available 001

Wait_coin 010

Check_amount 011

Dispense 100

Give_change 101

اگر جایی را به صورت reg تعریف کردم دلیلش این است که قرار است در بلاک always مقدار دهی بشود. مثلا برای بخش تعداد کالا ها که با نام product_x گذاشتم به صورت reg هست و در کلاک initial مقدار دهی می‌شوند و در آخر کار قرار هر دفعه اگر پول کافی بود از تعدادش کم بشود

و برای مقدار change و state و product برای اینکه نشان بدم در چه مرحله ای هستم output reg تعریف شده که مثل lcd عمل کند(همه گی به عنوان lcd).

در بخش بدنه کد از case استفاده کرده ام به جای if برای راحتی بیشتر از case استفاده شده است و به کیس کلی قرار گرفته برای هر استیت و داخل هر استیت اگر لازم بود که حالات مختلف رو چک بشود به case دیگه گذاشته شده تا چک شود

طبق منطقی که برای هر استیت توضیح داده ام تو ی if else های مجزا اینکه با چه شرایطی وارد استیت بعدی میشود مشخص کردم و به دکمه ریست هم در نظر گرفتم که به حالت صفر بر می‌گرداند.

Data

در این بخش قیمت های هر کالا مشخص شده براساس دسیمال در بخش پایین میزان کالا مشخص شده که به صورت reg تعریف شده به این دلیل که لازم است از مقادیر آن کم شود در بلاک always

در پایین آن استیت های ماشین مقدار دهی شده اند

```
input clk,
input reset,
input [1:0] coin,
input [1:0] select,
input vend,
output reg [7:0] total,
output reg [3:0] product,
output reg [7:0] change,
output reg [2:0] state
);

// Define product prices
parameter PRICE_A = 8'd10;
parameter PRICE_B = 8'd20;
parameter PRICE_C = 8'd30;
parameter PRICE_D = 8'd40;
//how many do we have
reg [7:0]product_0 = 5;
reg [7:0]product_1 = 4;
reg [7:0]product_2 = 2;
reg [7:0]product_3 = 7;

// Define states
parameter IDLE = 3'b000;
parameter AVAILABLE = 3'b001;
parameter WAIT_COIN = 3'b010;
parameter CHECK_AMOUNT = 3'b011;
parameter DISPENSE = 3'b100;
parameter OUT_OF_CHANGE = 3'b101;
```

Start

در بخش اولی اگر سیگنال reset فعال شد سریعاً به حالت ابتدایی برمیگردیم سپس در بخش بعدی برای استیت یک case قرار دادیم تا در صورت برابر بودن استیت فعلی با یکی از استیت ها وارد بخش مدنظر شویم سپس ابتدا در استیت ابتدایی قرار داریم در صورتی که به این حالت برگردیم باید همه چیز به صورت پایه قرار گیرند (دلیل اینکه product

```
total = 0;
product = 0;
change = 0;
end

always @(posedge clk or posedge reset) begin
    if (reset) begin
        state <= IDLE;
        total <= 0;
        product <= -1;
        change <= 0;
    end else begin
        case (state)
            IDLE: begin
                total <= 0;
                product <= -1;
                change <= 0;
                if (vend) begin
                    state <= AVAILABLE;
                end
            end
            AVAILABLE: begin
                case (select)
                    0 : if (product_0 > 0) state <= WAIT_COIN; else state <= IDLE;
                    1 : if (product_1 > 0) state <= WAIT_COIN; else state <= IDLE;
                    2 : if (product_2 > 0) state <= WAIT_COIN; else state <= IDLE;
                    3 : if (product_3 > 0) state <= WAIT_COIN; else state <= IDLE;
                endcase
            end
            WAIT_COIN: begin
                total <= total + coin;
                if (total <= PRICE_A) state <= CHECK_AMOUNT;
                else if (total <= PRICE_B) state <= CHECK_AMOUNT;
                else if (total <= PRICE_C) state <= CHECK_AMOUNT;
                else if (total <= PRICE_D) state <= CHECK_AMOUNT;
                else state <= OUT_OF_CHANGE;
            end
            CHECK_AMOUNT: begin
                if (total <= PRICE_A) state <= DISPENSE;
                else if (total <= PRICE_B) state <= DISPENSE;
                else if (total <= PRICE_C) state <= DISPENSE;
                else if (total <= PRICE_D) state <= DISPENSE;
                else state <= OUT_OF_CHANGE;
            end
            DISPENSE: begin
                total <= total - PRICE_A;
                total <= total - PRICE_B;
                total <= total - PRICE_C;
                total <= total - PRICE_D;
                product <= product + 1;
                change <= total;
                state <= IDLE;
            end
            OUT_OF_CHANGE: begin
                state <= IDLE;
            end
        endcase
    end
end
```

به صورت منفی ۱ هست این است که کد یکی از کالا ها ۰ است به همین دلیل حالت پایه را صفر در نظر گرفتم تا از میزان تعداد کالای ۰ کم نشود)

سپس در صورتی که کلید vend سیگنال ۱ بدهد فرایند و عملیات شروع می‌شود

```
WAIT_COIN: begin
  case (coin)
    0 : total <= total + 1;
    1 : total <= total + 10;
    2 : total <= total + 20;
    3 : total <= total + 50;
  endcase
  if (vend) state <= CHECK_AMOUNT;
end
```

```
CHECK_AMOUNT: begin
  case (select)
    0 : if (total >= PRICE_A) state <= DISPENSE; else state <= WAIT_COIN;
    1 : if (total >= PRICE_B) state <= DISPENSE; else state <= WAIT_COIN;
    2 : if (total >= PRICE_C) state <= DISPENSE; else state <= WAIT_COIN;
    3 : if (total >= PRICE_D) state <= DISPENSE; else state <= WAIT_COIN;
  endcase
end
```

```
DISPENSE: begin
  case (select)
    0 : begin
      product <= 0;
      change <= total - PRICE_A;
      product_0 = product_0 - 1;
    end
    1 : begin
      product <= 1;
      change <= total - PRICE_B;
```

states

در بخش بعدی اگر اگر شرط لازم

available که بودن کالا در ماشین

است برقرار بود وارد بخش بعدی شده

که منتظر می‌مانیم تا سکه ها به

اندازه وارد شوند سپس وقتی دکمه

vend را بزند یعنی پول کافی بوده و

وارد بعدی می‌شود اگر وند را بزند اما

پول کافی نباشد دوباره به استتیت

قبل باز می‌گردیم تا پول کافی وارد

شود(در اینجا ارور کمبود پول برای خرید کالا را در نظر

گرفتم) اگر میزان پول کافی بود وارد استتیت بعدی

شده و شروع به کم کردن از تعداد کالا، برگرداندن

میزان پول اضافه وارد شده در اینجا نیز نشان

می‌دهیم کالای خروجی چه چیزی بوده است

```
      change <= total - PRICE_A;
      product_0 = product_0 - 1;
    end
    1 : begin
      product <= 1;
      change <= total - PRICE_B;
      product_1 = product_1 - 1;
    end
    2 : begin
      product <= 2;
      change <= total - PRICE_C;
      product_2 = product_2 - 1;
    end
    3 : begin
      product <= 3;
      change <= total - PRICE_D;
      product_3 = product_3 - 1;
    end
  endcase
  state <= GIVE_CHANGE;
end

GIVE_CHANGE: begin
  change <= total;
  total <= 0;
  state <= IDLE;
end
endcase
end
```